

Security Monitoring and Incident Response Report

This report demonstrates the implementation of security monitoring and incident response through the setup of a basic security monitoring system using the Elastic Stack (ELK) for a specific use case, including detection rules, alert prioritization, and response procedures. Additionally, it documents an incident response scenario with classification, response steps, and lessons learned. All processes are documented with evidence of functionality.

1. Security Monitoring Setup

1.1 Environment

Tool: Elastic Stack (Elasticsearch, Logstash, Kibana)

Operating System: Ubuntu 20.04 LTS

Hardware: VM with 4 vCPUs, 16 GB RAM, 200 GB disk

Purpose: Monitor network and system logs to detect suspicious activity on a Linux server (192.168.1.100, previously identified in vulnerability scans).

1.2 Setup Steps

Install Elastic Stack: `sudo apt-get update`

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -  
echo "deb https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/  
sources.list.d/elastic-8.x.list
```

```
sudo apt-get update && sudo apt-get install -y elasticsearch logstash kibana
```

Configure Elasticsearch:

Edit `/etc/elasticsearch/elasticsearch.yml`: `network.host: 0.0.0.0`

`http.port: 9200`

Start Elasticsearch: `sudo systemctl enable elasticsearch`

```
sudo systemctl start elasticsearch
```

Configure Logstash:

Create `/etc/logstash/conf.d/syslog.conf` to ingest syslog data: `input {`

```
  syslog {  
    port => 514  
  }  
}  
output {  
  elasticsearch {  
    hosts => ["http://localhost:9200"]  
    index => "syslog-%{+YYYY.MM.dd}"  
  }  
}
```

Start Logstash: `sudo systemctl enable logstash`

```
sudo systemctl start logstash
```

Configure Kibana:

```
Edit /etc/kibana/kibana.yml:server.port: 5601  
server.host: "0.0.0.0"  
elasticsearch.hosts: ["http://localhost:9200"]
```

```
Start Kibana:sudo systemctl enable kibana  
sudo systemctl start kibana
```

Configure Syslog on Target Server:

```
On 192.168.1.100, configure rsyslog to forward logs to the ELK server (192.168.1.200):echo "**.*  
@192.168.1.200:514" | sudo tee -a /etc/rsyslog.conf  
sudo systemctl restart rsyslog
```

Access Kibana:

Open <http://192.168.1.200:5601> in a browser.

Create an index pattern (syslog-*) in Kibana under Stack Management > Index Patterns.

1.3 Use Case: Detecting Suspicious SSH Login Attempts

Scenario: Monitor for multiple failed SSH login attempts on the Linux server (192.168.1.100), which could indicate a brute-force attack.

Detection Rules:

Kibana Alert Rule:

Navigate to Kibana > Observability > Alerts > Create Rule.

Rule Type: Threshold.

Index: syslog-.*

Query: program:sshd AND "Failed password".

Threshold: Count >= 5 within 5 minutes.

Action: Log to console (for demonstration; in production, use email or webhook).

Name: SSH_BruteForce_Alert.

Logic: The rule triggers if five or more failed SSH login attempts are logged within a 5-minute window, indicating potential brute-force activity.

Alert Prioritization Process:

Severity: High

Justification: Brute-force attacks can lead to unauthorized access, especially if SSH credentials are weak or the server has known vulnerabilities (e.g., weak key exchange algorithms from prior scans).

Prioritization Criteria:

Impact: Compromise of SSH access could allow full server control.

Likelihood: High, given SSH's exposure on port 22 and common targeting in brute-force campaigns.

Context: Correlate with other indicators (e.g., source IP reputation, prior vulnerabilities) to escalate priority.

Escalation: Alerts are prioritized for immediate review by the incident response team if triggered, with secondary review for lower counts (e.g., 2–4 attempts).

Response Procedures:

Alert Review:

Check Kibana dashboard (Discover > syslog-*) for details on failed login attempts (source IP, username, timestamp).

Initial Containment:

Block the source IP on the firewall: `sudo ufw deny from <source_IP> to any port 22`

Investigation:

Verify if the targeted account exists and check for successful logins: `sudo grep "Accepted password" /var/log/auth.log`

Analyze source IP reputation using threat intelligence (e.g., OpenCTI, VirusTotal).

Mitigation:

Enforce stronger SSH configurations (e.g., key-based authentication, disable root login). `sudo sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config`
`sudo systemctl restart sshd`

Implement rate-limiting with tools like fail2ban.

Documentation:

Log the incident in a ticketing system with details (alert time, actions taken, outcome).

Evidence of Functionality:

Sample Alert Log (simulated): Timestamp: 2025-04-29 14:30:15

Rule: SSH_BruteForce_Alert

Condition: 6 failed SSH login attempts from 203.0.113.10

Action: Logged to console

Kibana Dashboard (simulated description): A syslog-* dashboard shows a time-series graph with 6 failed SSH attempts from 203.0.113.10 between 14:30:10 and 14:30:14 on 2025-04-29.
Firewall Rule (simulated):\$ sudo ufw status

Status: active

To	Action	From
--	-----	----
22	DENY	203.0.113.10

2. Incident Response Scenario

2.1 Scenario: Unauthorized SSH Access

Incident Description: On 2025-04-29 at 15:00, the SSH_BruteForce_Alert triggered, followed by a successful SSH login from the same source IP (203.0.113.10) to the account admin on 192.168.1.100. Logs indicate the attacker executed commands to download a malicious script.

2.2 Incident Classification

Severity: Critical

Justification: Unauthorized access to a server with critical services (e.g., HTTP, SMB) poses a severe risk of data theft, ransomware, or further network compromise.

Category: Unauthorized Access (MITRE ATT&CK T1078: Valid Accounts).

Impact: Potential compromise of the entire server, data exfiltration, or persistence mechanisms (e.g., backdoors).

Likelihood of Escalation: High, as the attacker has already gained access and executed commands.

2.3 Response Steps Taken

Detection and Alert:

Kibana alert triggered at 15:00 for 6 failed SSH attempts, followed by a successful login at 15:01.

Log entry:2025-04-29 15:01:10 sshd[1234]: Accepted password for admin from 203.0.113.10 port 54321 ssh2

Containment:

Blocked the source IP:sudo ufw deny from 203.0.113.10

Terminated the active SSH session:sudo pkill -u admin

Disabled the admin account:sudo passwd -l admin

Eradication:

Identified and removed the malicious script (/tmp/malware.sh):`sudo rm /tmp/malware.sh`

Scanned the system for additional malicious files using ClamAV:`sudo clamscan -r / --bell -i`

Reset SSH configurations to disable password authentication:`sudo sed -i 's/PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config`
`sudo systemctl restart sshd`

Recovery:

Restored the admin account with a new key-based authentication setup:`sudo passwd -u admin`
`sudo mkdir -p /home/admin/.ssh`
`sudo cp /path/to/new_key.pub /home/admin/.ssh/authorized_keys`
`sudo chown -R admin:admin /home/admin/.ssh`

Verified system integrity and service availability (Apache, SMB).

Post-Incident Analysis:

Analyzed logs to confirm no further unauthorized access.

Uploaded the malicious script's hash to OpenCTI (from prior implementation) to track related threats.

Documented the incident in a report (below).

2.4 Lessons Learned

Weak Authentication: Password-based SSH authentication enabled the brute-force attack. Key-based authentication should be mandatory.

Monitoring Gaps: The alert detected failed attempts but missed the successful login in real-time. Add a rule to detect successful logins from high-risk IPs.

New Rule (implemented post-incident):`Query: program:sshd AND "Accepted password" AND NOT src_ip:192.168.1.0/24`

Threshold: `Count >= 1`

Action: Alert to incident response team

Response Time: Manual IP blocking was effective but slow. Automate blocking with tools like fail2ban or IDS/IPS integration.

Documentation: Detailed logging of actions taken was critical for post-incident analysis and audit.

2.5 Evidence of Functionality

Incident Report (simulated): Incident ID: 8e5d2c1f-7b3a-4f9e-9d6c-4a2b8e7d0f1c
Date: 2025-04-29
Time: 15:00
Severity: Critical
Category: Unauthorized Access
Description: Successful SSH login by attacker (203.0.113.10) after brute-force attempt.
Actions Taken:

- Blocked IP (ufw deny)
- Terminated session (pkill)
- Disabled account (passwd -l)
- Removed malicious script (/tmp/malware.sh)
- Enforced key-based authentication

Status: Closed
Lessons Learned: Enforce key-based auth, add successful login alerts, automate blocking.

Log Evidence (simulated):
\$ sudo grep sshd /var/log/auth.log
2025-04-29 15:00:50 sshd[1234]: Failed password for admin from 203.0.113.10 port 54321 ssh2
2025-04-29 15:01:10 sshd[1234]: Accepted password for admin from 203.0.113.10 port 54321 ssh2

Kibana Alert (simulated): Alert: SSH_BruteForce_Alert
Timestamp: 2025-04-29 15:00:15
Details: 6 failed SSH attempts from 203.0.113.10

3. Conclusion

This project implemented a security monitoring system using the Elastic Stack to detect SSH brute-force attempts, with a clear detection rule, prioritization process, and response procedures. The incident response scenario demonstrated effective handling of an unauthorized SSH access incident, with containment, eradication, recovery, and lessons learned to improve future responses. All steps are supported by simulated evidence, ensuring a practical and documented approach to security monitoring and incident response.